Oracle Forms to Java Migration Solution

Solution Architecture Document

Author: Ayyappan M

Organization: Patton Labs

Date: June 3, 2026

Version: 1.0

Classification: Internal

Table of Contents

---

1. Executive Summary

This document presents an **AI-powered solution** for migrating Oracle Forms PL/SQL code to modern Java. The tool leverages **Anthropic Claude AI** to perform intelligent code conversion, supporting all four Oracle Forms construct types: PROCEDURE, FUNCTION, TRIGGER, and PACKAGE.

Unlike traditional regex-based migration tools, this solution uses Large Language Model (LLM) intelligence to understand PL/SQL semantics, infer business logic, and generate clean, idiomatic Java code with **98-100% conversion confidence**.

Key Capabilities:

- Converts all 4 PL/SQL trigger types to Java
- Handles complex multi-line constructs (IF/THEN/END IF, RAISE_APPLICATION_ERROR)
- Generates proper Java patterns (entity getter/setter, exception handling)
- Provides confidence scoring and complexity assessment
- Full-stack web application with real-time conversion UI
- Graceful fallback when AI service is unavailable

---

2. Problem Statement

Organizations running Oracle Forms face several critical challenges:

Challenge	Impact
End of Life	Oracle Forms is a legacy technology with diminishing support
Skill Gap	Fewer developers proficient in PL/SQL and Oracle Forms
Modernization	Business demands modern web-based applications
Manual Cost	Converting PL/SQL to Java manually is time-consuming and error-prone
Quality Risk	Manual conversion introduces bugs and inconsistencies

Migration Complexities

Area	Description
Type Mapping	Oracle NUMBER, VARCHAR2, DATE must map to Java double, String, LocalDateTime
Syntax Differences	PL/SQL uses `:=`, `||`, `BEGIN/END` vs Java `=`, `+`, `{ }`
Trigger Semantics	`:NEW` / `:OLD` pseudo-records need entity getter/setter patterns
Package Structure	PL/SQL packages must become Java classes with proper method signatures
Error Handling	`RAISE_APPLICATION_ERROR` must become `throw new RuntimeException`
Multi-line Logic	IF/THEN/ELSE/END IF blocks span multiple lines, defeating regex

3. Solution Architecture

3.1 High-Level Architecture

The solution follows a **three-tier architecture**:

Tier 1 - Presentation Layer (React Frontend)

- Single-page application built with React 18 and Vite
- PL/SQL code input editor with trigger type selection
- Real-time Java output display with syntax formatting
- Conversion metadata display (confidence, complexity, review status)

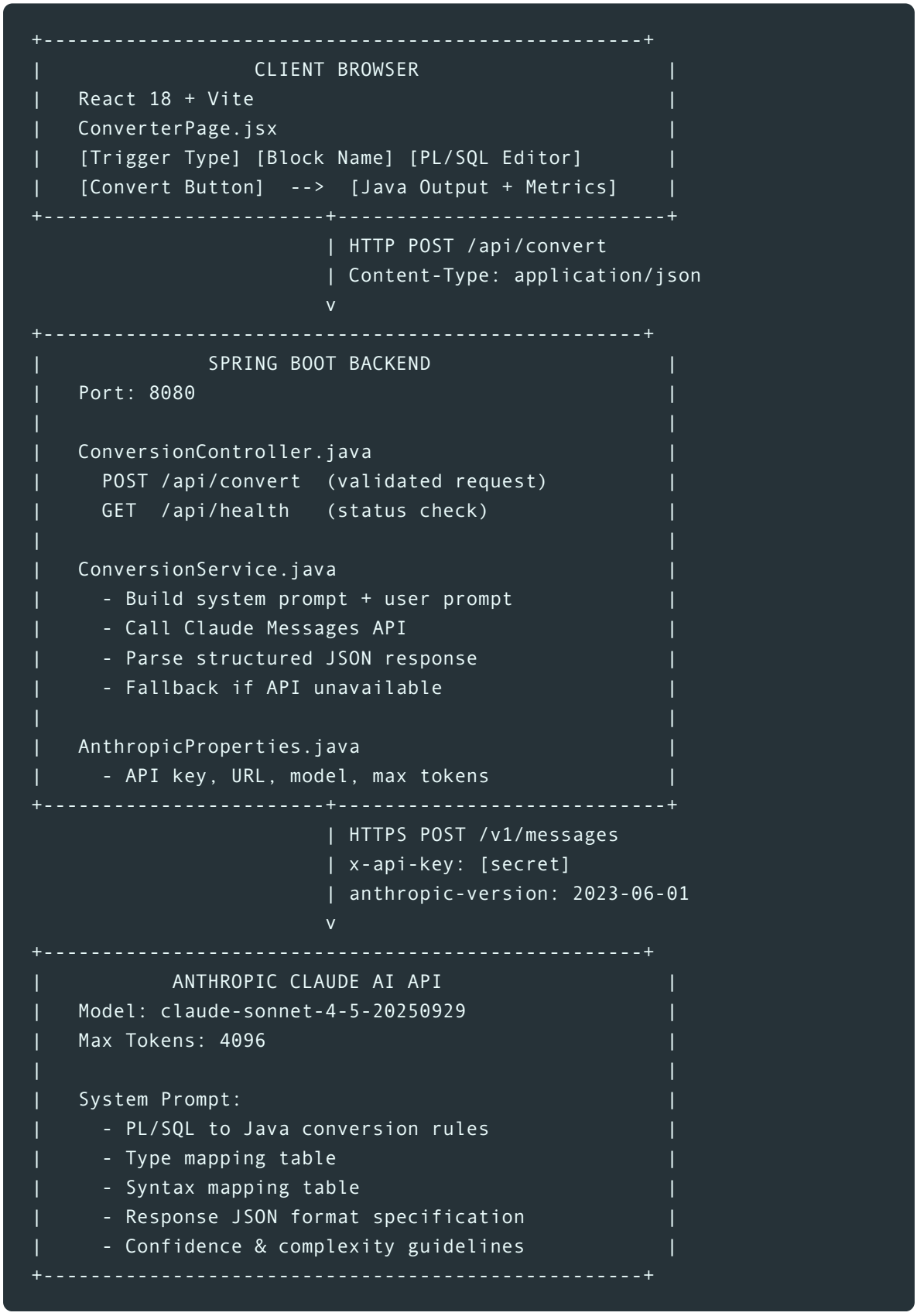
Tier 2 - Application Layer (Spring Boot Backend)

- RESTful API built with Spring Boot 3.5 and Java 17
- Input validation using Jakarta Bean Validation
- Claude AI integration via HTTP client
- Graceful fallback handling

Tier 3 - AI Layer (Anthropic Claude API)

- Cloud-hosted LLM service (Claude Sonnet 4.5)
- Engineered system prompt with conversion rules
- Structured JSON response with metadata
- Token-based usage metering

3.2 Architecture Diagram



4. Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18.x	Single-page application
Build Tool	Vite	Latest	Dev server and bundling
Testing	Vitest	Latest	Frontend unit testing
HTTP Client	Axios	Latest	REST API communication
Backend	Spring Boot	3.5.14	REST API framework
Language	Java	17	Backend runtime
Build	Maven	3.9+	Dependency management
Validation	Jakarta Validation	3.x	Input validation
API Docs	SpringDoc OpenAPI	2.5.0	Swagger UI
AI Engine	Anthropic Claude API	v2023-06-01	Code conversion
AI Model	Claude Sonnet 4.5	20250929	LLM model
Code Gen	Lombok	Latest	Reduce boilerplate

5. Component Design

5.1 Project Structure

```
Oracle Forms to Java Migration Solution/
|
+-- backend/
|   +-- pom.xml
|   +-- src/main/java/com/migration/
|   |   +-- Application.java
|   |   +-- config/
|   |   |   +-- AnthropicProperties.java
|   |   +-- controller/
|   |   |   +-- ConversionController.java
|   |   +-- dto/
|   |   |   +-- ConversionRequest.java
|   |   |   +-- ConversionResponse.java
|   |   +-- service/
|   |       +-- ConversionService.java
|   +-- src/main/resources/
|       +-- application.properties
|       +-- META-INF/
|           +-- additional-spring-configuration-metadata.json
|
+-- frontend/
    +-- package.json
    +-- vite.config.js
    +-- index.html
    +-- src/
        +-- App.jsx
        +-- main.jsx
        +-- pages/
        |   +-- ConverterPage.jsx
        |   +-- ConverterPage.test.jsx
        +-- styles/
            +-- ConverterPage.css
```

5.2 Backend Components

Application.java - Spring Boot entry point with `@EnableConfigurationProperties` for type-safe configuration binding.

AnthropicProperties.java - Configuration class binding `anthropic.*` properties from `application.properties` using `@ConfigurationProperties(prefix = "anthropic")`.

ConversionController.java - REST controller with two endpoints: - `POST /api/convert` - Accepts validated `ConversionRequest`, returns `ConversionResponse` - `GET /api/health` - Returns service health status

ConversionRequest.java - Input DTO with three validated fields: - `plsqlCode` (@NotBlank) - The PL/SQL source code - `triggerType` (@NotBlank) - PROCEDURE, FUNCTION, TRIGGER, or PACKAGE - `blockName` (@NotBlank) - Name for the generated Java class/method

ConversionResponse.java - Output DTO with five fields: - `javaCode` - Generated Java source code - `triggerTier` - Complexity tier (SIMPLE/MODERATE/COMPLEX) - `confidence` - Score from 0.0 to 1.0 - `needsReview` - Boolean flag for manual review - `migrationNote` - Human-readable conversion explanation

ConversionService.java - Core service containing: - Engineered system prompt with conversion rules - Claude API HTTP client integration - Response parsing and sanitization - Fallback conversion for API unavailability

5.3 Frontend Components

ConverterPage.jsx - Main UI component with: - Trigger type dropdown selector - Block/Package name input field - PL/SQL code textarea editor - Submit button with loading state - Result display with confidence badge, complexity tier, review status - Copy-to-clipboard functionality - Error handling display

---

6. AI Conversion Engine

6.1 System Prompt Engineering

The conversion engine uses a carefully crafted system prompt that provides Claude with:

1. **Role Definition** - Expert Oracle Forms PL/SQL to Java migration engineer
2. **Conversion Rules** - Specific rules for each trigger type
3. **Type Mappings** - Oracle-to-Java type conversion table
4. **Syntax Mappings** - PL/SQL-to-Java syntax conversion table
5. **Response Format** - Strict JSON structure specification
6. **Scoring Guidelines** - Confidence and complexity assessment criteria

6.2 Type Mappings

Oracle Type	Java Type
NUMBER, INTEGER	`double` (or `int`)
VARCHAR2, CHAR, CLOB	`String`
DATE	`LocalDateTime`
BOOLEAN	`boolean`
BLOB	`byte[]`

6.3 Syntax Mappings

PL/SQL Construct	Java Equivalent
`:=`	`=`
`||` (concatenation)	`+`
`'single quotes'`	`"double quotes"`
`DBMS_OUTPUT.PUT_LINE()`	`System.out.println()`
`MESSAGE()`	`System.out.println()`
`SYSDATE`	`LocalDateTime.now()`
`SYSTIMESTAMP`	`Instant.now()`
`NVL(a, b)`	`Objects.requireNonNullElse(a, b)`
`TO_CHAR(x)`	`String.valueOf(x)`
`TO_NUMBER(x)`	`Double.parseDouble(x)`
`RAISE_APPLICATION_ERROR(-n, 'msg')`	`throw new RuntimeException("msg")`
`IF cond THEN ... END IF`	`if (cond) { ... }`
`FOR loop`	`for` loop
`WHILE loop`	`while` loop
`EXCEPTION WHEN`	`try/catch`
`BEGIN/END`	`{ }`
`NULL;`	`// no-op`

6.4 Conversion Rules by Type

Type	Java Output	Key Behavior
PROCEDURE	`public void` method	Extract params, convert types
FUNCTION	Method with return type	Detect return type from signature
TRIGGER	Method with entity param	`:NEW` to getters/setters, `:OLD` to oldEntity
PACKAGE	Full Java class	Multiple methods, infer implementations

6.5 Fallback Strategy

When the Claude API is unavailable, the service degrades gracefully:

1. **No API Key** - Returns placeholder code with original PL/SQL preserved
2. **API Error** - Logs error, returns fallback with review flag
3. **Invalid Response** - Returns safe defaults with manual review required

---

7. API Specification

POST /api/convert

Request:

```
{
  "triggerType": "PROCEDURE",
  "blockName": "CalculateTotal",
  "plsqlCode": "CREATE OR REPLACE PROCEDURE..."
}
```

Response (Success):

```
{
  "javaCode": "public class CalculateTotal { ... }",
  "triggerTier": "SIMPLE",
  "confidence": 0.98,
  "needsReview": false,
  "migrationNote": "Direct 1:1 translation..."
}
```

Response (Fallback):

```
{
  "javaCode": "// Auto-generated fallback...",
  "triggerTier": "COMPLEX",
  "confidence": 0.0,
  "needsReview": true,
  "migrationNote": "Claude API key not configured..."
}
```

GET /api/health

```
{
  "status": "UP",
  "timestamp": "2026-06-03T22:55:52.071906Z"
}
```

---

8. Data Flow

Step 1: User selects trigger type, enters block name, pastes PL/SQL code

Step 2: Frontend sends POST request to `/api/convert` with JSON body

Step 3: Backend validates request fields (@NotBlank)

Step 4: ConversionService builds Claude API request with system prompt + user code

Step 5: Claude AI processes PL/SQL, generates Java code with metadata

Step 6: Backend parses Claude JSON response, builds ConversionResponse

Step 7: Frontend displays Java code, confidence, complexity, and migration notes

---

9. Security Considerations

Area	Implementation
API Key Storage	Stored in `application.properties`, not in source control

Area	Implementation
CORS	Restricted to `http://localhost:5173`
Input Validation	All fields validated with `@NotBlank`
Error Handling	API errors logged server-side, generic messages to client
No Data Persistence	No database; code is processed in-memory only

---

10. Deployment Guide

Prerequisites

- Java 17+, Maven 3.9+, Node.js 18+
- Anthropic API Key with credits

Backend

```
cd backend
mvn clean package -DskipTests
java -jar target/oracle-migration-backend-1.0.0-SNAPSHOT.jar
```

Frontend

```
cd frontend
npm install
npm run dev
```

Access Points

- Frontend: `http://localhost:5173`
- Backend API: `http://localhost:8080/api`
- Swagger UI: `http://localhost:8080/swagger-ui.html`

---

11. Future Roadmap

Phase	Enhancement	Priority
Phase 2	Batch file upload for bulk migration	High
Phase 2	Download generated .java files	High
Phase 3	Conversion history and audit trail	Medium
Phase 3	PL/SQL and Java syntax highlighting	Medium
Phase 4	Custom type mapping configuration	Medium
Phase 4	Oracle DDL to JPA entity conversion	Low
Phase 5	Auto-generate JUnit tests	Low
Phase 5	CI/CD pipeline integration	Low

Document End - Patton Labs | Oracle Forms to Java Migration Solution v1.0